

Examination Portal

File-to-File Flow Reference

Every arrow in this document represents a direct code dependency or call: an import, a function call, an API request, or a DB operation. Files are shown by their short path relative to the project root.

Colour / Style	Meaning
FROM / TO in blue mono	Source file → destination file
→ in red	Direction of the call or import
Step label (grey bar)	Groups arrows that belong to the same action

App Bootstrap

How the React app starts up and wraps every page with auth context.

Browser loads index.html → React entry point			
FROM		TO	What is passed / triggered
frontend/index.html	→	frontend/src/main.jsx	Browser executes main.jsx as the JS entry point
frontend/src/main.jsx	→	frontend/src/context/AuthContext.jsx	Wraps the entire app in so every page can access user state
frontend/src/main.jsx	→	frontend/src/App.jsx	Mounts inside AuthProvider; App owns all routes
frontend/src/App.jsx	→	frontend/src/context/AuthContext.jsx	useAuth() — reads current user and loading state to decide route redirects
frontend/src/context/AuthContext.jsx	→	frontend/src/services/api.js	Calls api.post('/auth/login') and api.post('/auth/register')
frontend/src/services/api.js	→	backend: AuthController.java	HTTP POST /api/auth/login or /api/auth/register with JSON body

Route protection — PrivateRoute in App.jsx			
FROM		TO	What is passed / triggered
frontend/src/App.jsx	→	frontend/src/pages/Login.jsx	Route /login renders Login page (redirects to dashboard if already logged in)
frontend/src/App.jsx	→	frontend/src/pages/Register.jsx	Route /register renders Register page
frontend/src/App.jsx	→	frontend/src/pages/admin/*	All /admin/* routes wrapped in PrivateRoute role='ADMIN'
frontend/src/App.jsx	→	frontend/src/pages/student/*	All /student/* routes wrapped in PrivateRoute role='STUDENT'

Shared Auth Flow (Register & Login)

Both admin and student use the same register and login pages. The only difference is the role value selected.

Register — frontend

FROM		TO	What is passed / triggered
pages/Register.jsx	→	components/auth/RegisterComponent.jsx	Passes formData, loading, onSubmit, onChange as props; component renders the form UI
pages/Register.jsx	→	context/AuthContext.jsx	Calls register(formData) from useAuth() hook
context/AuthContext.jsx	→	services/api.js	api.post('/auth/register', userData)

Register — backend

FROM		TO	What is passed / triggered
services/api.js	→	AuthController.java	POST /api/auth/register → @PostMapping('/register')
AuthController.java	→	AuthService.java	authService.register(request)
AuthService.java	→	UserRepository.java	existsByUsername(), existsByEmail(), save(user)
AuthService.java	→	JwtUtil.java	generateToken(username, role) — creates signed JWT
UserRepository.java	→	MySQL: users table	INSERT INTO users (username, email, password, role, ...)
AuthController.java	→	pages/Register.jsx	Returns AuthResponse { token, id, username, email, role }
pages/Register.jsx	→	App.jsx (navigate)	navigate('/admin/dashboard') or navigate('/student/dashboard') based on role

Login — frontend

FROM		TO	What is passed / triggered
pages/Login.jsx	→	components/auth/LoginComponent.jsx	Passes formData, loading, onSubmit, onChange; renders form with 'Username or Email' field
pages/Login.jsx	→	context/AuthContext.jsx	Calls login(formData) from useAuth() hook
context/AuthContext.jsx	→	services/api.js	api.post('/auth/login', credentials)

Login — backend

FROM		TO	What is passed / triggered
services/api.js	→	AuthController.java	POST /api/auth/login
AuthController.java	→	AuthService.java	authService.login(request)
AuthService.java	→	UserRepository.java	findByUsername() OR findByEmail() — supports both login methods
AuthService.java	→	CustomUserDetailsService.java	AuthenticationManager.authenticate() triggers loadUserByUsername()
CustomUserDetailsService.java	→	UserRepository.java	findByUsername() OR findByEmail() to load UserDetails
AuthService.java	→	JwtUtil.java	generateToken(username, role)
AuthController.java	→	pages/Login.jsx	Returns AuthResponse { token, id, username, email, role }

FROM		TO	What is passed / triggered
pages/Login.jsx	→	App.jsx (navigate)	navigate to /admin/dashboard or /student/dashboard based on role

Every subsequent API call: services/api.js interceptor reads token from localStorage and adds 'Authorization: Bearer ' header automatically.

JWT validation on every protected request			
FROM		TO	What is passed / triggered
services/api.js	→	JwtRequestFilter.java	Intercepts every HTTP request, extracts JWT from Authorization header
JwtRequestFilter.java	→	JwtUtil.java	extractUsername(jwt), validateToken(jwt, userDetails)
JwtRequestFilter.java	→	CustomUserDetailsService.java	loadUserByUsername(username) to build UserDetails
JwtRequestFilter.java	→	SecurityConfig.java	Sets authentication in SecurityContextHolder; SecurityConfig enforces @PreAuthorize roles

Scenario 1: Admin File Flows

Admin Dashboard

FROM		TO	What is passed / triggered
pages/admin/AdminDashboard.jsx	→	components/Layout.jsx	Wraps page in sidebar + header shell with role='ADMIN'
pages/admin/AdminDashboard.jsx	→	components/admin/AdminDashboardComponent.jsx	Passes stats object { totalUsers, totalStudents, totalSubjects, totalQuestions, totalTests }
pages/admin/AdminDashboard.jsx	→	services/api.js	GET /admin/users, /admin/subjects, /admin/questions, /admin/tests (parallel)
services/api.js	→	AdminController.java	getAllUsers(), getAllSubjects(), getAllQuestions(), getAllTests()
AdminController.java	→	UserService.java	getAllUsers()
AdminController.java	→	SubjectService.java	getAllSubjects()
AdminController.java	→	QuestionService.java	getAllQuestions()
AdminController.java	→	TestService.java	getAllTests()
UserService.java	→	UserRepository.java	findAll() → SELECT * FROM users
SubjectService.java	→	SubjectRepository.java	findAll() → SELECT * FROM subjects
QuestionService.java	→	QuestionRepository.java	findAll() → SELECT * FROM questions
TestService.java	→	TestRepository.java	findAll() → SELECT * FROM tests

Subject Management

FROM		TO	What is passed / triggered
pages/admin/SubjectManagement.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/SubjectManagement.jsx	→	components/admin/SubjectManagementComponent.jsx	Passes subjects[], showForm, formData, onSubmit, onEdit, onDelete, onCancel
pages/admin/SubjectManagement.jsx	→	services/api.js	GET /admin/subjects on load
pages/admin/SubjectManagement.jsx	→	services/api.js	POST /admin/subjects (create), PUT /admin/subjects/{id} (update), DELETE /admin/subjects/{id}
services/api.js	→	AdminController.java	createSubject(), updateSubject(), deleteSubject()
AdminController.java	→	SubjectService.java	createSubject(subject), updateSubject(id, subject), deleteSubject(id)
SubjectService.java	→	SubjectRepository.java	save(subject) → INSERT/UPDATE subjects deleteById(id) → DELETE subjects
SubjectRepository.java	→	MySQL: subjects table	INSERT / UPDATE / DELETE INTO subjects

Question Management

FROM		TO	What is passed / triggered
pages/admin/QuestionManagement.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/QuestionManagement.jsx	→	components/admin/QuestionManagementComponent.jsx	Passes questions[], subjects[], formData, editingQuestion, onSubmit, onEdit, onDelete
pages/admin/QuestionManagement.jsx	→	services/api.js	GET /admin/questions + GET /admin/subjects on load (parallel)
pages/admin/QuestionManagement.jsx	→	services/api.js	POST /admin/questions (create), PUT /admin/questions/{id} (update), DELETE /admin/questions/{id}
services/api.js	→	AdminController.java	createQuestion(), updateQuestion(), deleteQuestion()
AdminController.java	→	QuestionService.java	createQuestion(q), updateQuestion(id, q), deleteQuestion(id)
QuestionService.java	→	QuestionRepository.java	save(question) → INSERT/UPDATE questions deleteById(id)
QuestionRepository.java	→	MySQL: questions table	INSERT / UPDATE / DELETE INTO questions (links to subjects via subject_id FK)

Test Management

FROM		TO	What is passed / triggered
pages/admin/TestManagement.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/TestManagement.jsx	→	components/admin/TestManagementComponent.jsx	Passes tests[], subjects[], questions[], showForm, showQuestionSelector, selectedTest, selectedQuestions, handlers
pages/admin/TestManagement.jsx	→	services/api.js	GET /admin/tests + /admin/subjects + /admin/questions on load (parallel)
pages/admin/TestManagement.jsx	→	services/api.js	POST /admin/tests (create test)
pages/admin/TestManagement.jsx	→	services/api.js	POST /admin/tests/{testId}/questions (add question IDs list)
pages/admin/TestManagement.jsx	→	services/api.js	DELETE /admin/tests/{id} (delete test)
services/api.js	→	AdminController.java	createTest(), addMultipleQuestionsToTest(), deleteTest()
AdminController.java	→	TestService.java	createTest(test, adminId), addMultipleQuestionsToTest(testId, questionIds), deleteTest(id)
TestService.java	→	TestRepository.java	save(test) → INSERT INTO tests
TestService.java	→	QuestionRepository.java	findById(questionId) for each question being added
TestService.java	→	MySQL: test_questions table	INSERT INTO test_questions (test_id, question_id, question_order) per question
TestRepository.java	→	MySQL: tests table	INSERT / DELETE INTO tests

User Management

FROM		TO	What is passed / triggered
pages/admin/UserManagement.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/UserManagement.jsx	→	components/admin/UserManagementComponent.jsx	Passes users[], roleFilter, onRoleFilterChange, onDelete
pages/admin/UserManagement.jsx	→	services/api.js	GET /admin/users on load
pages/admin/UserManagement.jsx	→	services/api.js	DELETE /admin/users/{id} on delete button
services/api.js	→	AdminController.java	getAllUsers(), deleteUser(id)
AdminController.java	→	UserService.java	getAllUsers(), deleteUser(id)
UserService.java	→	UserRepository.java	findAll() → SELECT * FROM users deleteById(id) → DELETE (cascades to attempts, answers)
UserRepository.java	→	MySQL: users table	SELECT / DELETE FROM users (ON DELETE CASCADE propagates to exam_attempts, student_answers)

Admin Reports

FROM		TO	What is passed / triggered
pages/admin/AdminReports.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/AdminReports.jsx	→	components/admin/AdminReportsComponent.jsx	Passes studentReport, subjectReport, activeTab, onTabChange
pages/admin/AdminReports.jsx	→	services/api.js	GET /admin/reports/students + GET /admin/reports/subjects (parallel)
services/api.js	→	AdminController.java	getAllStudentsReport(), getSubjectWiseReport()
AdminController.java	→	ReportService.java	getAllStudentsReport(), getSubjectWiseReport()
ReportService.java	→	ExamAttemptRepository.java	Aggregate queries: JOIN exam_attempts + users + subjects → avg score, pass rate
ExamAttemptRepository.java	→	MySQL: exam_attempts table	SELECT with GROUP BY student_id / subject_id to compute aggregates

Admin Profile

FROM		TO	What is passed / triggered
pages/admin/AdminProfile.jsx	→	components/Layout.jsx	Sidebar shell
pages/admin/AdminProfile.jsx	→	components/admin/AdminProfileComponent.jsx	Passes profile object and formatDate helper
pages/admin/AdminProfile.jsx	→	services/api.js	GET /admin/profile
services/api.js	→	AdminController.java	getProfile(Authentication authentication)
AdminController.java	→	UserService.java	getUserByUsername(username) — username extracted from JWT
UserService.java	→	UserRepository.java	findByUsername(username) → SELECT * FROM users WHERE username = ?
UserRepository.java	→	MySQL: users table	SELECT single user row

Scenario 2: Student File Flows

Student Dashboard

FROM		TO	What is passed / triggered
pages/student/StudentDashboard.jsx	→	components/Layout.jsx	Sidebar shell with role='STUDENT'
pages/student/StudentDashboard.jsx	→	components/student/StudentDashboardComponent.jsx	Passes stats { totalAttempts, averageScore, highestScore, lowestScore, passedCount, failedCount, recentAttempts } and tests[]
pages/student/StudentDashboard.jsx	→	services/api.js	GET /student/performance + GET /student/tests (parallel)
services/api.js	→	StudentController.java	getMyPerformance(), getAvailableTests()
StudentController.java	→	ReportService.java	getStudentPerformance(studentId) — aggregate stats
StudentController.java	→	TestService.java	getActiveTests()
ReportService.java	→	ExamAttemptRepository.java	findCompletedAttemptsByStudent(id) → aggregate score stats
TestService.java	→	TestRepository.java	findByIsActive(true) → SELECT * FROM tests WHERE is_active = 1

Available Tests

FROM		TO	What is passed / triggered
pages/student/AvailableTests.jsx	→	components/Layout.jsx	Sidebar shell
pages/student/AvailableTests.jsx	→	components/student/AvailableTestsComponent.jsx	Passes tests[], onStartTest handler
pages/student/AvailableTests.jsx	→	services/api.js	GET /student/tests on load
pages/student/AvailableTests.jsx	→	services/api.js	POST /student/exams/start/{testId} when 'Start Test' clicked
services/api.js	→	StudentController.java	getAvailableTests(), startExam(testId, authentication)
StudentController.java	→	ExamService.java	startExam(studentId, testId)
ExamService.java	→	UserRepository.java	findById(studentId) — fetch student entity
ExamService.java	→	TestRepository.java	findById(testId) — fetch test entity
ExamService.java	→	ExamAttemptRepository.java	save(attempt) → INSERT INTO exam_attempts (status='IN_PROGRESS')
ExamAttemptRepository.java	→	MySQL: exam_attempts table	INSERT row: student_id, test_id, start_time, status=IN_PROGRESS
pages/student/AvailableTests.jsx	→	pages/student/ExamPage.jsx	navigate('/student/exam/:testId', { state: { attemptId } })

Exam Page

FROM		TO	What is passed / triggered
pages/student/ExamPage.jsx	→	components/Layout.jsx	Sidebar shell
pages/student/ExamPage.jsx	→	components/student/ExamPageComponent.jsx	Passes test, questions[], currentQuestionIndex, answers[], timeRemaining, marksPerQuestion, handlers
pages/student/ExamPage.jsx	→	services/api.js	GET /student/exams/active/{testId} — check for existing attempt if no attemptId in state
pages/student/ExamPage.jsx	→	services/api.js	GET /student/exams/attempts/{attemptId} — load attempt details and test info
pages/student/ExamPage.jsx	→	services/api.js	GET /student/exams/{attemptId}/questions — load question list (no correct answers)
services/api.js	→	StudentController.java	getActiveAttempt(), getAttemptDetails(), getExamQuestions()
StudentController.java	→	ExamService.java	getActiveAttempt(studentId, testId), getAttemptById(id)
ExamService.java	→	ExamAttemptRepository.java	findByStudentIdAndTestIdAndStatus(), findById()
ExamAttemptRepository.java	→	MySQL: exam_attempts + test_questions + questions	SELECT attempt JOIN test JOIN questions (correct_option excluded in response)

Per-answer save (every time student selects an option)

FROM		TO	What is passed / triggered
components/student/ExamPageComponent.jsx	→	pages/student/ExamPage.jsx	onAnswerSelect(questionId, optionNum) callback fires
pages/student/ExamPage.jsx	→	services/api.js	POST /student/exams/answer?attemptId={id} with { questionId, selectedOption }
services/api.js	→	StudentController.java	saveAnswer(answerRequest, attemptId)
StudentController.java	→	ExamService.java	saveAnswer(attemptId, answerRequest)
ExamService.java	→	StudentAnswerRepository.java	findByExamAttemptIdAndQuestionId() — upsert: update if exists, insert if new
ExamService.java	→	QuestionRepository.java	findById(questionId) — compare selectedOption with correctOption
StudentAnswerRepository.java	→	MySQL: student_answers table	INSERT or UPDATE student_answers (exam_attempt_id, question_id, selected_option, is_correct)

Exam submission (Submit button or timer expiry)

FROM		TO	What is passed / triggered
components/student/ExamPageComponent.jsx	→	pages/student/ExamPage.jsx	onSubmit(autoSubmit) callback fires
pages/student/ExamPage.jsx	→	services/api.js	POST /student/exams/submit with { attemptId, answers[] }
services/api.js	→	StudentController.java	submitExam(SubmitExamRequest)
StudentController.java	→	ExamService.java	saveAnswer() for each answer in list, then submitExam(attemptId)

FROM		TO	What is passed / triggered
ExamService.java	→	StudentAnswerRepository.java	findByExamAttemptId() — fetch all saved answers
ExamService.java	→	ExamAttemptRepository.java	save(attempt) → UPDATE exam_attempts: score, correct/wrong/unanswered, end_time, status=COMPLETED
ExamAttemptRepository.java	→	MySQL: exam_attempts table	UPDATE SET score, correct_answers, wrong_answers, unanswered, end_time, status='COMPLETED'
pages/student/ExamPage.jsx	→	pages/student/PerformanceReports.jsx	navigate('/student/performance') after successful submit

Test Report

FROM		TO	What is passed / triggered
pages/student/TestReport.jsx	→	components/Layout.jsx	Sidebar shell
pages/student/TestReport.jsx	→	components/student/TestReportComponent.jsx	Passes attempt, answers[], marksPerQuestion, showEmailModal, sendingEmail, handlers
pages/student/TestReport.jsx	→	services/api.js	GET /student/exams/attempts/{attemptId} — attempt details
pages/student/TestReport.jsx	→	services/api.js	GET /student/exams/{attemptId}/answers — full answer list with question data
services/api.js	→	StudentController.java	getAttemptDetails(attemptId), getAttemptAnswers(attemptId)
StudentController.java	→	ExamService.java	getAttemptById(id), getAttemptAnswers(attemptId)
ExamService.java	→	ExamAttemptRepository.java	findById(attemptId) → SELECT attempt with nested test + subject
ExamService.java	→	StudentAnswerRepository.java	findByExamAttemptId(id) → SELECT answers with nested question data
StudentAnswerRepository.java	→	MySQL: student_answers + questions	SELECT student_answers JOIN questions (includes correctOption for review)

Email results button

FROM		TO	What is passed / triggered
components/student/TestReportComponent.jsx	→	pages/student/TestReport.jsx	onEmailResults() callback fires
pages/student/TestReport.jsx	→	services/api.js	POST /student/email-results/{attemptId}
services/api.js	→	StudentController.java	emailResults(attemptId, authentication)
StudentController.java	→	ExamService.java	getAttemptById(attemptId) — verify student owns the attempt
StudentController.java	→	EmailService.java	sendExamResult(student.email, attempt)
EmailService.java	→	Gmail SMTP (external)	JavaMailSender.send() — delivers result email to student's registered address

Performance Reports

FROM		TO	What is passed / triggered
pages/student/PerformanceReports.jsx	→	components/Layout.jsx	Sidebar shell
pages/student/PerformanceReports.jsx	→	components/student/PerformanceReportsComponent.jsx	Passes performance, subjectPerformance, attempts[], onViewReport
pages/student/PerformanceReports.jsx	→	services/api.js	GET /student/performance + /student/performance/subjects + /student/exams/attempts (parallel)
services/api.js	→	StudentController.java	getMyPerformance(), getSubjectWisePerformance(), getMyAttempts()

FROM		TO	What is passed / triggered
StudentController.java	→	ReportService.java	getStudentPerformance(id), getSubjectWisePerformance(id)
StudentController.java	→	ExamService.java	getCompletedAttempts(studentId)
ReportService.java	→	ExamAttemptRepository.java	Aggregate queries grouped by subject → avg/min/max score, attempt count
ExamService.java	→	ExamAttemptRepository.java	findCompletedAttemptsByStudent(id) → SELECT WHERE status='COMPLETED'
ExamAttemptRepository.java	→	MySQL: exam_attempts + tests + subjects	SELECT with joins and aggregates for charts and stats
components/student/PerformanceReportsComponent.jsx	→	pages/student/TestReport.jsx	onViewReport(attemptId) → navigate('/student/report/:attemptId')

Student Profile

FROM		TO	What is passed / triggered
pages/student/Profile.jsx	→	components/Layout.jsx	Sidebar shell
pages/student/Profile.jsx	→	context/AuthContext.jsx	useAuth() — reads user from context (used as fallback identifier)
pages/student/Profile.jsx	→	components/student/ProfileComponent.jsx	Passes profile object and formatDate helper
pages/student/Profile.jsx	→	services/api.js	GET /student/profile
services/api.js	→	StudentController.java	getProfile(Authentication authentication)
StudentController.java	→	UserService.java	getUserByUsername(username) — username from JWT
UserService.java	→	UserRepository.java	findByUsername(username) → SELECT * FROM users WHERE username = ?
UserRepository.java	→	MySQL: users table	SELECT single user row

Complete File Index

Every file in the project and its role in the flow.

File	Layer	Role
frontend/index.html	Entry	HTML shell — loads main.jsx
frontend/src/main.jsx	Entry	React root render; wraps App in AuthProvider
frontend/src/App.jsx	Router	All route definitions; PrivateRoute role guard
frontend/src/services/api.js	HTTP	Axios instance; attaches JWT header; handles 401 redirect
frontend/src/context/AuthContext.jsx	State	Global auth state; login(), register(), logout(); stores token in localStorage
frontend/src/components/Layout.jsx	UI	Sidebar + header shell shared by all pages
pages/Login.jsx	Page	State + logic for login form; delegates UI to LoginComponent
pages/Register.jsx	Page	State + logic for register form; delegates UI to RegisterComponent
pages/admin/AdminDashboard.jsx	Page	Loads stats from 4 API calls; renders AdminDashboardComponent
pages/admin/SubjectManagement.jsx	Page	CRUD state for subjects; renders SubjectManagementComponent
pages/admin/QuestionManagement.jsx	Page	CRUD state for questions; renders QuestionManagementComponent
pages/admin/TestManagement.jsx	Page	CRUD state for tests + question selector; renders TestManagementComponent
pages/admin/UserManagement.jsx	Page	Loads users; delete handler; renders UserManagementComponent
pages/admin/AdminReports.jsx	Page	Loads student + subject reports; renders AdminReportsComponent
pages/admin/AdminProfile.jsx	Page	Loads admin profile; renders AdminProfileComponent
pages/student/StudentDashboard.jsx	Page	Loads performance stats + tests; renders StudentDashboardComponent
pages/student/AvailableTests.jsx	Page	Loads tests; starts exam attempt; navigates to ExamPage
pages/student/ExamPage.jsx	Page	Manages exam state, timer, per-answer save, submission; renders ExamPageComponent
pages/student/TestReport.jsx	Page	Loads attempt + answers; email trigger; renders TestReportComponent
pages/student/PerformanceReports.jsx	Page	Loads 3 data sources; navigates to TestReport; renders PerformanceReportsComponent
pages/student/Profile.jsx	Page	Loads student profile; renders ProfileComponent
components/auth/LoginComponent.jsx	UI	Pure login form UI (label: Username or Email)
components/auth/RegisterComponent.jsx	UI	Pure registration form UI with role dropdown
components/admin/AdminDashboardComponent.jsx	UI	Stat cards + quick action links
components/admin/SubjectManagementComponent.jsx	UI	Subject table + add/edit form
components/admin/QuestionManagementComponent.jsx	UI	Question table + add/edit form with all 4 options

File	Layer	Role
components/admin/TestManagementComponent.jsx	UI	Test table + create form + question selector panel
components/admin/UserManagementComponent.jsx	UI	User table with role filter buttons
components/admin/AdminReportsComponent.jsx	UI	Two-tab report table (student-wise, subject-wise)
components/admin/AdminProfileComponent.jsx	UI	Profile info grid display
components/student/StudentDashboardComponent.jsx	UI	6 stat cards + recent attempts list
components/student/AvailableTestsComponent.jsx	UI	Test cards with Start Test button
components/student/ExamPageComponent.jsx	UI	Question view, options, timer, navigation panel
components/student/TestReportComponent.jsx	UI	Result summary + Q&A; analysis + email modal
components/student/PerformanceReportsComponent.jsx	UI	Stat cards + Recharts bar charts + attempt history table
components/student/ProfileComponent.jsx	UI	Profile info grid with avatar initial
AuthController.java	Controller	POST /auth/register, POST /auth/login
AdminController.java	Controller	All /admin/* endpoints; @PreAuthorize ADMIN
StudentController.java	Controller	All /student/* endpoints; @PreAuthorize STUDENT
AuthService.java	Service	register() and login() logic; username-or-email lookup
UserService.java	Service	getAllUsers(), getUserByUsername(), updateUser(), deleteUser()
SubjectService.java	Service	createSubject(), updateSubject(), deleteSubject()
QuestionService.java	Service	createQuestion(), updateQuestion(), deleteQuestion()
TestService.java	Service	createTest(), addMultipleQuestionsToTest(), getActiveTests()
ExamService.java	Service	startExam(), saveAnswer(), submitExam(), getAttemptById()
ReportService.java	Service	getStudentPerformance(), getSubjectWisePerformance(), getAllStudentsReport()
EmailService.java	Service	sendExamResult() via JavaMailSender → Gmail SMTP
JwtUtil.java	Security	generateToken(), extractUsername(), validateToken()
JwtRequestFilter.java	Security	Intercepts every request; validates JWT; sets SecurityContext
CustomUserDetailsService.java	Security	loadUserByUsername() — tries username then email
SecurityConfig.java	Security	Configures permitted routes, filter chain, password encoder
UserRepository.java	Repository	JPA: users table — findByUsername, findByEmail, existsByUsername
SubjectRepository.java	Repository	JPA: subjects table
QuestionRepository.java	Repository	JPA: questions table — findBySubjectId
TestRepository.java	Repository	JPA: tests table — findByIsActive

File	Layer	Role
ExamAttemptRepository.java	Repository	JPA: exam_attempts — findById, findCompletedAttemptsByStudent
StudentAnswerRepository.java	Repository	JPA: student_answers — findById, findByIdAndQuestionId
MySQL: users	DB	id, username, email, password, role, is_active, created_at
MySQL: subjects	DB	id, name, description, is_active
MySQL: questions	DB	id, subject_id, question_text, option1–4, correct_option, marks, difficulty
MySQL: tests	DB	id, title, subject_id, duration_minutes, total_marks, passing_marks, created_by
MySQL: test_questions	DB	test_id, question_id, question_order (join table)
MySQL: exam_attempts	DB	student_id, test_id, start_time, end_time, score, correct/wrong/unanswered, status
MySQL: student_answers	DB	exam_attempt_id, question_id, selected_option, is_correct

End of Document